



PAL INFORMATICA

Besturingssystemen - Sessie 1

Yarne Ramakers, Jonas Couwberghs – 12/03/2026

Inhoud



1. De basis van xv6
2. De Makefile
3. System Calls
4. Hoe bereid je je voor op de test?
5. Vragen over de Labos
6. Voorbeeld examen



DE BASIS VAN XV6

1. Structuur van xv6, waar staat wat?



`user/`: Code draait in **User Mode**.

- ▶ User programs:

- `ls.c`
- `cat.c`
- ...

- ▶ Libraries:

- `ulib.c`
- `printf.c`
- ...

- ▶ Interface:

- `user.h`
- Definieert wat een user programma mag aanroepen

vs.

`kernel/`: Code draait in **Kernel Mode**.

- ▶ Procesbeheer

- ▶ Geheugenmanagement

- ▶ Hardwareinteractie

- ▶ Implementaties van system calls:

- `sys_read`
- `sys_fork`
- ...

2. De User-Kernel Grens



Een programma in `user/` roept een functie uit `user.h` op:

De functie is een library call:

- ▶ Blijft volledig binnen **User Space**
- ▶ Voorbeelden: `strcpy`, `atoi`, `memset`
- ▶ Gewone C-functies die op de stack van het proces draaien

vs.

De functie is een system call:

- ▶ Vormt de brug naar de **Kernel Space**
- ▶ Voorbeelden: `fork`, `write`, `open`
- ▶ Gebruiken een speciale CPU-instructie (`ecall` op RISC-V) om de kernel om hulp te vragen

3. De onderdelen van de Kernel



Veel bestanden in `kernel/` → Moeilijk om te weten wat juist wat doet

Categorie	Bestanden	Wat doet het?
Boot & Start	<code>entry.S</code> , <code>start.c</code> , <code>main.c</code>	De allereerste instructies die de CPU uitvoert
System Calls	<code>syscall.c</code> , <code>sysproc.c</code> , <code>sysfile.c</code>	De “dispatcher” die user requests afhandelt
Resources	<code>proc.c</code> , <code>kalloc.c</code> , <code>vm.c</code>	Beheer van processen en het RAM-geheugen
Filestysteem	<code>fs.c</code> , <code>bio.c</code> , <code>file.c</code> , <code>log.c</code>	Hoe data op de “harde schijf” (<code>fs.img</code>) staat
Hardware	<code>console.c</code> , <code>uart.c</code> , <code>virtio_disk.o</code>	Praten met het scherm toetsenbord, etc.

3. De onderdelen van de Kernel

Overige categorieën



Categorie	Bestanden	Wat doet het?
Interrupt Handling	<code>trap.c</code>	Beslist of een interrupt van de hardware komt, een fout is (page fault), of een system call
Assembly & Low-level	<code>trampoline.S</code> , <code>swtch.S</code>	Kritieke code voor context switches en het opslaan van CPU registers



DE MAKEFILE

1. Kernel en User space in de Makefile



Bovenaan de Makefile vinden we het volgende terug:

```
U=user  
K=kernel
```

En overall in de Makefile zie je dingen zoals:

```
$U/ulib.o  
$K/pipe.o
```

- ▶ `$U/`: We verwijzen naar een bestand/component uit de User Space `user/`
- ▶ `$K/`: We verwijzen naar een bestand/component uit de Kernel Space `kernel/`

2. User Programs



Alle user programs moeten in de makefile worden opgelijst in UPROGS

- ▶ Je maakt een user program `user/hello.c`
- ▶ Je voegt dit in de Makefile toe in UPROGS als `$U/_hello\`
 - Vergeet zeker niet de underscore
- ▶ De Makefile weet hierdoor impliciet dat hij het bestand `user/hello.c` moet compilen naar `user/_hello`
- ▶ De Makefile voegt deze dan toe in het filesystem van xv6 dat zich in het bestand `fs.img` bevindt:

```
fs.img: mkfs/mkfs README $(UPROGS) $(EXTRA_FS_FILES)
mkfs/mkfs fs.img README $(UPROGS) $(EXTRA_FS_FILES)
```

- ▶ Je kan nu het programma oproepen vanuit xv6 als `hello`

```
UPROGS=\
$U/_cat\
$U/_echo\
$U/_forktest\
$U/_grep\
$U/_hello\
$U/_init\
$U/... \
$U/_wc\
$U/_zombie\
$U/_halt\
$U/_logstress\
$U/_forphan\
$U/_dorphan\
```

3. Libraries



Alle library bestanden moeten in de makefile worden opgelijst in `ULIB`

- ▶ Je maakt een nieuwe library functie
- ▶ Je voegt de header van de functie toe in `user.h`
- ▶ In een al bestaand bestand
 - Je moet niets aanpassen in de Makefile
- ▶ In een nieuw bestand
 - Je voegt je bestand `user/mylib.c` toe aan `ULIB` als `$U/mylib.o`

```
ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/mylib.o
```

4. De Kernel



In de Makefile zie je een lange lijst onder `OBJS = \`. Dit zijn de onderdelen die samen de Kernel vormen.

- ▶ Als je een nieuw bestand aanmaakt voor iets in de kernel (bijv. `kernel/mysyscall.c`), voeg je dit hier ook toe (als `$K/mysyscall.o`)
- ▶ De Makefile linkt deze `.o` bestanden samen met behulp van de linker naar `kernel/kernel`

```
OBJS = \  
$K/entry.o \  
$K/start.o \  
$K/console.o \  
$K/printf.o \  
$K/... \  
$K/pipe.o \  
$K/exec.o \  
$K/sysfile.o \  
$K/kernelvec.o \  
$K/plic.o \  
$K/virtio_disk.o \  
$K/mysyscall.o
```



SYSTEM CALLS

1. De System Call: Stap voor Stap



Hoe gaat xv6 van een `write()` in user space naar de kernel en weer terug?

1. De System Call: Stap voor Stap

Stap 1 & 2: User Space en de “Trap”



- ▶ **De C-Wrapper (`usys.s`):** Een assembly-script zet het unieke syscall-nummer in register `a7` en de argumenten in registers `a0` tot `a5`.
- ▶ **De Trap (`ecall`):** Deze speciale RISC-V instructie gooit een hardware-interrupt, pauzeert user mode, en geeft de controle aan de kernel.
- ▶ **De Trampoline (`trampoline.s`):** Slaa alle huidige CPU-registers op in het **trapframe** van het proces (zodat we later precies weten waar we waren) en wisselt naar de veilige Kernel Stack.

2. De System Call: In de Kernel



2. De System Call: In de Kernel

Stap 3 & 4: Dispatching en Uitvoering



- ▶ **De Dispatcher (`syscall.c`):** De kernel kijkt in het opgeslagen `a7` register, gebruikt dit als index voor een tabel (`syscalls[]`), en roept de bijbehorende C-functie aan (bijv. `sys_write`).
- ▶ **Uitvoering (`sysproc.c` / `sysfile.c`):** De kernel doet het zware werk (zoals praten met de harde schijf of geheugen toewijzen).
- ▶ **Het Resultaat:** De return value (bijv. de uitgelezen bytes of foutcode `-1`) wordt in het `a0` register van het **trapframe** geplaatst.

2. De System Call: In de Kernel

Stap 5: Terug naar User Space



- ▶ **Afronden (`trap.c`):** De functie `usertrapret()` bereidt de CPU voor op de terugkeer naar de lagere privileges.
- ▶ **Herstellen (`trampoline.s`):** Alle user-registers worden netjes teruggezet vanuit het trapframe.
- ▶ **Return (`sret`):** Deze instructie dropt de privileges terug naar User Mode en hervat het programma exact op de instructie volgend op de originele `ecall`.

4. Een System Call Implementeren: User Space



Hoe maken we onze nieuwe syscall zichtbaar voor programma's?

► **1. Definieer de interface (`user/user.h`)**

- Voeg de C-declaratie toe, bv: `int my_syscall(int);`
- **Waarom?** De C-compiler heeft een prototype nodig, anders weten user-programma's (zoals `main.c`) niet dat de functie bestaat.

► **2. Genereer de assembly-brug (`user/usys.pl`)**

- Voeg toe: `entry("my_syscall");`
- **Waarom?** Dit Perl-script genereert tijdens het compileren `usys.s`. Dit bestand bevat de magische assembly-code die het juiste ID in het `a7`-register plaatst en de `ecall`-instructie uitvoert.

5. Een System Call Implementeren



De kernel moet weten wat hij met het inkomende verzoek moet doen.

► 3. Geef de syscall een ID (`kernel/syscall.h`)

- Voeg toe: `#define SYS_my_syscall 22`
- **Waarom?** Registers werken met getallen, niet met namen. Dit is de unieke identifier van je syscall.

► 4. Koppel het ID aan de code (`kernel/syscall.c`)

- Definieer prototype: `extern uint64 sys_my_syscall(void);`
- Voeg toe aan de array: `[SYS_my_syscall] sys_my_syscall`
- **Waarom?** De algemene `syscall()` dispatcher gebruikt het a7-register als index voor deze array map. Hierdoor weet hij precies welke C-functie hij moet aanroepen.

6. Een System Call Implementeren: De Logica



Het echte werk gebeurt in functies zoals `sysproc.c` of `sysfile.c`.

- ▶ **5. Schrijf de implementatie (`sys_my_syscall(void)`)**
 - **Waarom `void`?** De functie neemt **geen** C-argumenten aan! De werkelijke argumenten zitten in de opgeslagen CPU-registers (het trapframe).
- ▶ **6. Haal argumenten veilig op**
 - Gebruik `argint()`, `argaddr()` (voor pointers/`uint64`), of `argstr()`.
 - **Waarom?** De kernel mag user-pointers nooit blindelings vertrouwen. Deze functies controleren of het geheugenadres wel echt van de user is en halen de waarden veilig uit de registers (`a0`, `a1`, etc.).
 - Je krijgt toegang tot het huidige proces via `myproc()`.

7. Optioneel: Process Bookkeeping



Soms moet je data onthouden over een proces (bv. hoe vaak een syscall is aangeroepen).

- ▶ **7. Breid de Process Control Block uit (`kernel/proc.h`)**
 - Voeg je variabele toe aan `struct proc`.
 - **Let op:** Gebruik `uint64` voor geheugenadressen (het is een 64-bit OS).
- ▶ **8. Initialiseer je variabelen (`kernel/proc.c`)**
 - Zet je nieuwe veld op `0` of `null` in de `allocproc()` functie.
 - **Waarom extreem belangrijk?** C wist geheugen niet automatisch! Als je dit niet doet, bevat jouw nieuwe variabele “garbage data” (willekeurige eentjes en nulletjes die toevallig nog in het RAM stonden).



HOE BEREID JE JE VOOR OP DE TEST?

Vorbereiden op de test



- ▶ Herbekijk de voorbije labos
- ▶ Je mag alles wat je wil meenemen
 - MAAK EEN SAMENVATTING
 - ▶ Stappenplan over hoe je een Syscall maakt (zoals in de voorgaande slides)
 - ▶ Stappenplan over hoe je een user program maakt
 - ▶ Stappenplan over hoe je een user library functie bijmaakt
 - ▶ Welk bestand in de kernel heeft welke functie?
 - Zo vindt je sneller terug in welk bestand je mogelijks moet zijn om iets aan te passen.
 - ▶ Een overzicht van de dingen die je voor de labos hebt moeten aanpassen



VRAGEN OVER DE LABOS

Ingestuurde vragen



► Geen

Overige vragen



Heeft iemand toch nog vragen over de voorbije 3 labos?



VOORBEELD EXAMEN

Vraag 1



In deze oefening breid je de “interrupt counting” functionaliteit van Lab 3 uit.

Code skeleton. Je krijgt de referentieoplossing van Lab 3 als uitgangspunt. De kern van de implementatie bevindt zich in de volgende functies:

- ▶ `sys_printinterrupts` in `kernel/sysproc.c`
- ▶ `devintr` in `kernel/trap.c`
- ▶ `scheduler` in `kernel/proc`

Vraag 1

Vervolg opgave



Momenteel ziet de uitvoer van het `printirq` userprogramma er als volgt uit:

```
init: starting sh
$ printirq
      CPU0    CPU1    CPU2
TMR:   26     26     26
UART:   9      5     13
DISK:  17     12      9
```

Vraag 1

Vervolg opgave



Je taak in deze opdracht is om deze uitvoer uit te breiden met het **aantal system calls per CPU**. Merk op: in tegenstelling tot `getnumsyscalls` in Lab 2, vragen we *niet* het aantal system calls per proces, maar het totale aantal system calls dat elke CPU heeft verwerkt, ongeacht welk proces de system call heeft uitgevoerd. De uitvoer van `printirq` zou er na jouw aanpassingen als volgt uit moeten zien:

```
$ printirq
```

	CPU0	CPU1	CPU2
TMR:	4	4	4
UART:	8	3	2
DISK:	17	17	6
SYS:	7	38	0

Vraag 1

Stap 1



1. Kijk hoe system calls en andere exceptions momenteel worden geïdentificeerd in de functie `usertrap` in `kernel/trap.c`.

Vraag 1

Stap 2



2. Pas `kernel/proc.h` aan om een extra counter toe te voegen die *per CPU* het aantal system calls bijhoudt. Zorg ervoor dat deze counter op nul wordt geïnitieerd, net zoals de bestaande counters in de modeloplossing.

Vraag 1

Stap 3



3. Implementeer de telling van system-call gebeurtenissen per CPU-core in de functie `usertrap` in `kernel/trap.c` (niet elders!). **Tips:**
- ▶ Bekijk de functie `devintr` in de modeloplossing om te begrijpen hoe je de huidige CPU kunt identificeren.
 - ▶ Voor system-call traps moet je *alle* system calls tellen, ongeacht hun geldigheid of effect op het proces.

Vraag 1

Stap 4



4. Breid de functie `sys_printinterrupts` in `kernel/sysproc.c` uit om de nieuwe telling te rapporteren. Gebruik hierbij de identifier `sys`, zoals in het voorbeeld.

Let op: Zorg voor correcte formattering; onjuiste output kan de test laten falen!

Vraag 1

Stap 5



5. Test je oplossing met `make test-irq`.

Vraag 2



In deze oefening moet je een **nieuwe system call** implementeren met de volgende signatuur:

```
int syscallfilter(int num);
```

Wanneer deze nieuwe system call wordt aangeroepen met een geldige parameter `num`, zullen de volgende aanroepen naar de system call die overeenkomt met `num` **geblokkeerd worden**. Deze filter is enkel van toepassing op het **huidige proces** en zijn **kinderen** (i.e., m.zgn. “child processes”). Merk op dat we geen unblock-functie aanbieden: zodra een system call is geblokkeerd, kan deze niet meer worden gedeblokkeerd. Op deze manier kunnen (child) processen in een afgeschermd “sandbox”-omgeving draaien, vergelijkbaar met de `seccomp` functionaliteit in real-world Linux.

Vraag 2

Voorbeeld



Het volgende xv6 programma mag alleen “Allowed” afdrukken:

```
int main()
{
    printf("Allowed\n");
    syscallfilter(SYS_write);
    if (fork() > 0) {
        printf("Not allowed - parent\n");
    } else {
        printf("Not allowed - child\n");
    }
    return 0;
}
```

Vraag 2

Vervolg opgave



We hebben dit en andere xv6 gebruikersprogramma's voor jou beschikbaar gesteld in de bestanden `user/testfilter1.c` tot `user/testfilter5.c`. Je kan deze testprogramma's gebruiken om je implementatie van `syscallfilter` te valideren m.b.v. `make test-syscall`. Het is **niet** de bedoeling dat je deze testprogramma's aanpast, aangezien we je oplossing zullen testen met onze eigen ongemodificeerde versies van de testprogramma's.

Vraag 2

Datastructuur



Elk xv6-proces moet zijn *eigen* system-call-filter informatie bijhouden. We zullen dit efficiënt op 1 als de system call met nummer `i` is toegestaan, en op 0 als deze geblokkeerd is. Je hoeft de benodigde bitmanipulaties *niet* zelf te implementeren; in plaats daarvan moet je de types en functies gebruiken die zijn gedeclareerd in `kernel/bitmap.h`:

- ▶ Het C-datatype `bitmap_t` is een abstracte representatie van de onderliggende bitmap. Je mag deze *nooit* rechtstreeks bewerken, maar alleen doorgeven aan de volgende functies.
- ▶ `bitmap_t init_bitmap(void)` maakt een nieuwe `bitmap_t` bitmap aan, waarbij alle bits op 1 staan (d.w.z. dat alle system calls initieël zijn toegestaan).

Vraag 2

Datastructuur vervolg



- ▶ `bitmap_t clear_bit(bitmap_t bitmap, uint64 num)`: maakt en retourneert een *nieuwe* bitmap afgeleid van de gegeven bitmap, maar waarbij de bit met nummer `num` op 0 staat. Je kan deze dus gebruiken om de system call met nummer `num` als geblokkeerd te markeren.
- ▶ `uint is_bit_set(bitmap_t bitmap, uint64 num)`: retourneert 1 (true) als de bit met nummer `num` gezet is in de gegeve bitmap, en 0 (false) als die op 0 staat. Je kan deze functie dus gebruiken om te testen of de system call met nummer `num` is toegestaan.

Vraag 2

Stap 1



Zorg dat xv6 een `bitmap_t` bijhoudt voor elk proces. Zorg ervoor dat je de `bitmap_t` correct initialiseert d.m.v. de functie `init_bitmap` en dat child processen de `bitmap_t` waarde van hun parent correct overerven.

Vraag 2

Stap 2



Pas de xv6 kernel code aan zodat system call met nummer 25 de functie `uint64 sys_syscallfilter(void)` aanroept.

Vraag 2

Stap 3



Voeg een (user-space) system call wrapper `syscallfilter` toe.

Vraag 2

Stap 4



Implementeer de system call in `uint64 sys_syscallfilter(void)` in het bestand `kernel/syscallfilter.c`. De system call moet eerst het argument `num` ophalen. Als dit een geldig system-call nummer is, moet de functie `sys_syscallfilter` de `bitmap_t` van het huidige proces bijwerken en 0 retourneren (i.e., success). Als `num` *geen* geldige system-call nummer is, moet de functie `sys_syscallfilter` de filter van het huidige proces onveranderd laten en `-1` retourneren (i.e., failure). Om te controleren of de system-call nummer geldig is, mag je aannemen dat system calls aaneengesloten zijn genummerd van 1 tot en met de waarde van de laatste system call.

Vraag 2

Stap 5



Implementeer ten slotte de kernfunctionaliteit: controleer telkens wanneer een `system call` wordt gedaan of deze is toegestaan door de `bitmap_t` van het huidige proces. Zo niet, beëindig dan het proces met `kill`.

Vraag 2

Stap 6



Valideer je oplossing met `make test-syscall`.